

Project Team Formation Portal

Project Proposal for UCS503P

Submitted to:

Thapar Institute of Engineering and Technology

Shreyansh

Akshin Gulati

Chandan Bansal

Ayush Shree Sonkar

August 19, 2026

1 Higher-order goal

This project aims to improve the process of academic project team formation by reducing the time, effort, and uncertainty involved in finding suitable teammates. The immediate engineering objective is to build a deployable web platform through which students and teachers can discover projects, identify suitable students, verify relevant skills, and form teams in a structured manner.

2 Time-to-value

- We will deliver meaningful increments quickly by first implementing the core workflow: authentication → profile → project discovery → join/invite → team formation.
- We will prioritize fast feedback through short iterations and early validation with representative student and teacher users.
- Success is defined by what can be delivered and measured in the first iteration, then improved using user feedback and testing.
- CI-backed automation will be introduced early so each major change can be built and tested consistently.

3 Problem Statement

College students frequently face difficulty forming project teams, particularly when projects require specific technical skills, academic criteria, or a balanced combination of roles. Common pain points include:

- Fragmentation: team formation is often handled through WhatsApp groups, personal communication, spreadsheets, or informal classroom discussions.
- Poor skill visibility: students have no structured way to discover students with specific skills such as Python, React, SQL, or Machine Learning.
- Unstructured recruitment: invitations and join requests are difficult to track when handled through multiple communication channels.
- Unverified skills: listed skills may not accurately represent a student's ability, making selection difficult.
- Limited supervision: teachers need a unified way to create or approve projects, monitor teams, and review participation.

Why this matters now (contextual relevance): As project-based learning becomes increasingly common in engineering education, a structured team-formation workflow can reduce recruitment effort and help students form teams based on skills and project requirements rather than only on existing social connections.

4 Proposed Solution

4.1 Overview

Build a web-based “Project Team Formation Portal” with the following components:

- Student profile system: academic information, skills, skill ratings, certificates, GitHub, LinkedIn, resume, and overall rating.
- Student discovery: search and filter students based on skills and academic criteria.
- Project marketplace: students and teachers create projects with descriptions, required skills, team size, deadlines, and eligibility criteria.
- Team recruitment: students request to join projects, while project leaders and teachers can invite students.
- Skill verification test: project creators can optionally define questions, time limit, marks, and passing score to assess applicants.
- Collaboration layer: formed teams receive private group chat, calendar events, reminders, and notifications.
- Teacher dashboard: project approval/rejection, project creation, invitations, supervision, and monitoring.
- Admin dashboard: user, teacher, project, department, announcement, and system-statistics management.

4.2 Core workflow

1. A student or teacher creates a project and specifies requirements such as skills, minimum CGPA, branch/year, and team size.
2. Students search for projects or suitable students using filters.
3. A student requests to join, or the project leader/teacher sends an invitation.
4. If enabled, the applicant completes a skill verification test before the request is considered.
5. The system checks eligibility and records the application/test result; the project leader or teacher accepts or rejects the applicant.
6. After team formation, members receive team chat, calendar events, reminders, and project information.

4.3 Operational constraints for an educational setting

- Keep the platform usable on low-to-mid range devices through responsive web design.
- Avoid dependence on advanced research algorithms; focus on reliable workflow, search, eligibility checks, and skill verification.
- Use common web technologies and managed services so the system can be deployed for a college pilot.
- Keep student data limited to information relevant to project formation and collaboration.

5 Solution Approach

This is an engineering course project, so the focus is on workflow correctness, usability, role-based access, database design, testing, and deployability rather than novel algorithmic research.

5.1 Skill verification and eligibility assistance

- Project creators can define minimum CGPA, required skills, branch/year restrictions, team-size limits, and whether a test is required.
- Skill verification tests will use objective questions initially and automatically calculate scores.
- The system will store test attempts, scores, and pass/fail outcomes for the relevant project.
- Test results support the project leader/teacher's decision but are not necessarily the only selection criterion.
- Eligibility rules will be enforced by the backend so users cannot bypass project requirements through the frontend.

5.2 Web architecture

- Frontend: responsive React interface for students, teachers, and administrators.
- Backend API: Node.js with Express.js for authentication, authorization, profiles, projects, recruitment, eligibility, tests, and notifications.
- Database: MySQL for users, profiles, skills, projects, teams, requests, invitations, tests, results, and events.
- Authentication: secure password hashing and role-based authorization for Student, Teacher, and Admin roles.

5.3 CI/CD and fast delivery enabler

- CI/CD will automatically build the application and run tests and linting on changes.
- Git and GitHub with feature branches and pull requests will coordinate development.
- Staging deployment will be used before final production deployment.

6 Evaluation Criterion

We define success using measurable metrics tied to the core team-formation workflow.

6.1 Primary evaluation metric

- Time-to-team-formation (TTF): time between a student's first project search/request action and the point at which the student is accepted into a project team.
- Target: demonstrate a measurable reduction in team-formation effort/time compared with the existing informal process during the pilot.
- Attribution: measured using database timestamps for project discovery, request/invitation, test completion where applicable, and acceptance.

6.2 Secondary evaluation metrics

- Recruitment completion rate: percentage of recruitment attempts resulting in a successfully formed team.
- Search effectiveness: percentage of users who can find a suitable student/project using filters.
- Skill verification usefulness: project leader/teacher rating of whether test results helped evaluate applicants.

- User clarity: student-reported ease of finding a suitable project/team using a simple 1–5 question.
- Reliability: availability of login, search, project creation, and recruitment workflows during the pilot.
- Workflow correctness: percentage of tested join/invite/eligibility scenarios producing expected results.

6.3 Pilot validation plan

- Recruit approximately 10–20 student users and 2–5 teacher/admin users.
- Create representative projects with different skill, CGPA, branch/year, and team-size requirements.
- Measure the core workflow from project discovery through team acceptance and collect short usability feedback.
- Compare the observed workflow with the existing informal process through short interviews or a baseline questionnaire.

7 Scalability

The proposition should scale in both theoretical and practical senses.

Use indexed database fields for common searches and pagination for student/project lists.

Separate frontend and backend components so each can be maintained and scaled independently.

Use efficient filtering and role-based queries for larger numbers of students, projects, and recruitment requests.

Deploy using common web hosting, a managed MySQL database, and CI/CD for staging and production.

8 Engine availability heuristic

A mature set of engines and services is available to implement the system as a web-based educational project:

- React and JavaScript for responsive frontend development.
- Node.js and Express.js for REST APIs and backend workflow logic.
- MySQL for relational data storage.
- Secure password hashing and token/session-based authentication.
- GitHub for source control and collaboration.
- Automated testing and CI runners for repeatable validation.
- Common web hosting and managed database services for deployment.

We will use these mature components to focus on workflow design, correctness, usability, and measurement rather than inventing new infrastructure.

9 Project scope and deliverables

9.1 Initial deliverable

- Student/Teacher/Admin login and role-based access.
- Student profile with basic information and skills.
- Project creation with basic requirements.

- Student/project search and filtering.
- Join request and invitation workflow.
- Initial database schema and REST APIs.
- CI: build, linting, and backend unit tests.

9.2 Subsequent deliverables

- GitHub, LinkedIn, resume, certificates, ratings, and expanded profile fields.
- Project eligibility rules and optional skill verification tests.
- Team chat, calendar, reminders, and notifications.
- Teacher dashboard for approval and supervision.
- Admin dashboard for management and statistics.
- Responsive UI refinement and dark mode.
- Integration/security testing, documentation, and final deployment.

10 Risks and mitigations

- Privacy and student data: minimize collected data and restrict information according to user roles.
- Unfair or misleading skill assessment: use transparent objective tests and treat test scores as supporting evidence rather than the only criterion.
- Low adoption: keep search, join, invitation, and project creation workflows simple.
- Incorrect authorization: enforce role and project-membership permissions on the backend.
- Evaluation ambiguity: instrument timestamps and define clear workflow states before development.
- Integration issues: use Git branches, pull requests, CI tests, and staging deployment throughout development.
- Scope expansion: prioritize the core team-formation workflow and treat advanced features as later deliverables.

11 Summary

This project proposes a deployable web platform for structured academic project team formation. It addresses the fragmented process of finding teammates by combining student profiles, skill-based discovery, project search, join requests, invitations, configurable eligibility criteria, and optional skill verification tests. It meets Software Engineering objectives by emphasizing time-to-value through rapid iterations, CI/CD-supported development, measurable evaluation criteria, role-based access, database-backed workflows, testing, and scalability readiness. The system remains focused on practical engineering and does not depend on advanced research algorithms.